# CMPE3209 - M1 Program Önerisi

Ali Bal | 5001230006 | alibal@ogr.iu.edu.tr | 2 Kasım 2026

## Adım 0 - Kişiye özel parametreler

Tam numara 5001230006 > 2147483647. Yönergedeki son 9 hane kuralıyla 001230006 sayısal olarak SID=1230006 olur. d, D2 ve S tam öğrenci numarasından hesaplanır.

| | |
|---|---|
| **d / D2 / S** | d = 6; D2 = 06; S = 5+0+0+1+2+3+0+0+0+6 = 17 |
| **N** | 24 + (6 mod 25) = 30 eleman; 30 × 4 = 120 byte |
| **Cache** | d=6: 64 byte / 16 byte line / direct mapping / 4 line / set yok |
| **Write policy** | 6 çift: write-back; write miss için write-allocate (varsayım) |
| **k / Interrupt** | (17 mod 7)+3 = 6; 17 tek: I/O |
| **f** | 1.0 + 0.1 × 6 = 1.6 GHz (varsayım) |
| **T1 / T2** | T1=1 cycle; T2=20+5×(6 mod 4)=30 cycle (varsayım) |
| **Nc** | 2 + (17 mod 7) = 5 çekirdek (varsayım) |
| **Görev parametresi** | P=50+(1230006 mod 10)=56; P ve üstü geçer |
| **Adres alanları** | 32 bit: word=log2(16)=4, line=log2(4)=2, tag=32-4-2=26 bit; s=28. Toplam=32 bit. |

## Görev, tasarım ve ISA

G2 Sınav notu dağılımı. Notları öğrenci numaramdan LCG ile üretip global grades dizisinde tutuyorum. İkinci geçişte notları beş aralıkta sayıp toplamı ve geçen kişi sayısını buluyorum. Tamsayı ortalamayı hesapladıktan sonra üçüncü geçişte mark_below fonksiyonuyla ortalamanın altındaki notları OR 0x100 ile işaretliyorum. Sonuçları simülatörde bellekten okuyabilmek için global değişkenlerde saklıyorum.

ISA: RV32IM / ILP32. Sabit 32 bit instruction formatları elle kodlama için uygun; M uzantısı çarpma/bölme içeriyor. Clang 18.1.6 (Zig 0.13.0), C99 -O0; Unicorn 2.1.4. Git: https://github.com/AliBal7/cmpe3209-proje (özel), etiket M1.

YZ beyanı: Kodun denetlenmesi ve derleme hatalarının giderilmesinde ChatGPT/Codex desteği kullanıldı. Sonuçlar RV32IM belleği ve bağımsız hesapla karşılaştırıldı.

# C99 kaynak kodu ve gereksinim eşlemesi

main.c | 39 satır | Kütüphane çağrısı, float ve dinamik bellek yok

```c
/* Ali Bal, 5001230006, G2 */
const int SID = 1230006;
enum { N = 30 };
int grades[N];
int bucket[5], mean, below_count, pass_count;

int mark_below(int value, int average)
{
    if (value < average)
        value = value | 0x100;
    return value;
}

int main(void)
{
    unsigned int x = (unsigned int)SID;
    int i, sum = 0, pass = 50 + SID % 10;
    for (i = 0; i < N; i++) {
        x = 1664525u * x + 1013904223u;
        grades[i] = (int)((x >> 16) % 101u);
    }
    for (i = 0; i < N; i++) {
        int v = grades[i];
        sum = sum + v;
        if (v < 50) bucket[0]++;
        else if (v < 65) bucket[1]++;
        else if (v < 75) bucket[2]++;
        else if (v < 85) bucket[3]++;
        else bucket[4]++;
        if (v >= pass) pass_count++;
    }
    mean = sum / N;
    for (i = 0; i < N; i++) {
        int v = grades[i];
        grades[i] = mark_below(v, mean);
        if (grades[i] & 0x100) below_count++;
    }
    return 0;
}
```

Y1 | 4, 18-21, 22-31, 33-37: Global int grades[30]; üretme, sayma ve işaretleme geçişleri.

Y2 | 9, 25-30, 36: if ile ortalama, not aralığı ve geçme koşulları.

Y3 | 7-12, 35: mark_below iki parametre alır, işaretli notu döndürür.

Y4 | 19-20, 24, 10, 36: Toplama, çarpma, shift, OR ve AND kullanılır.

Y5 | 20, 35: Üretilen ve işaretlenen değer grades[i] adresine yazılır.

Y6 | 2, 16-17: const int SID; LCG başlangıcını ve geçme notunu etkiler.

Y7 | 5, 25-32, 36: bucket, mean, below_count, pass_count global sonuçlardır.

Kernel: C 7-12 ve 35; i=0, v=7. 0x000102D4 jalr çağrısından 0x0001004C ret'e 20 dinamik instruction. lw, sw, ori/addi, blt ve call/return içerir. Ayrıntılı liste tutanakta.

Doğrulanan çıktılar: bucket={14,1,7,0,8}, mean=51, below_count=14, pass_count=15. Toplam=1553, tamsayı ortalama=1553/30=51. Kanıt: evidence/verification.json.